Unidad 3

# Conjuntos (Set)

Colecciones sin valores repetidos con new Set, sus métodos add, delete, has, clear y size, y el operador spread para quitar duplicados y unir arrays.

Un **conjunto** (`Set`) es una colección parecida a un array con una regla añadida: **no admite valores duplicados**. No hay que comprobar nada antes de añadir; si el valor ya está, el conjunto simplemente lo ignora.

```
const asistentes = new Set();

asistentes.add('ana');
asistentes.add('luis');
asistentes.add('ana');       // ya estaba: no pasa nada

console.log(asistentes);     // → Set(2) { 'ana', 'luis' }
console.log(asistentes.size);   // → 2
```

Esa es la razón para usar un `Set` en lugar de un array: cuando lo que importa es *qué* valores hay y no cuántas veces aparecen ni en qué posición.

## Creación

Se crea con `new Set()`. Opcionalmente se le pasa algo **iterable**, como un array, una cadena o un mapa, y el conjunto se rellena con sus elementos, descartando los repetidos:

```
const vacio = new Set();

const numeros = new Set([1, 2, 2, 3, 3, 3]);
console.log(numeros);         // → Set(3) { 1, 2, 3 }

const letras = new Set('banana');
console.log(letras);          // → Set(3) { 'b', 'a', 'n' }
```

## Recorrido

Los conjuntos no tienen índices: `conjunto[0]` es `undefined`. El bucle natural para recorrerlos es `for … of`, que entrega los valores en el orden en que se insertaron:

```
const numeros = new Set([10, 20, 30]);

for (const n of numeros) {
  console.log(n);
}
// → 10
// → 20
// → 30
```

## Añadir, eliminar y consultar

`add(valor)` añade un elemento y devuelve el propio conjunto, lo que permite **encadenar** llamadas:

```
const pares = new Set();
pares.add(2).add(4).add(6);
console.log(pares.size);      // → 3
```

`delete(valor)` elimina un elemento y devuelve `true` si estaba y `false` si no; `clear()` vacía el conjunto entero:

```
pares.delete(4);              // → true   Set(2) { 2, 6 }
pares.delete(5);              // → false  el 5 no estaba

pares.clear();
console.log(pares.size);      // → 0
```

El tamaño se consulta con la propiedad `size`, no con `length` (que en un `Set` es `undefined`). Y `has(valor)` comprueba si un valor está, con la misma comparación estricta que `includes` en los arrays:

```
const dias = new Set(['lun', 'mar', 'mié']);

dias.has('mar');              // → true
```

```
dias.has('dom');              // → false
dias.size;                    // → 3
dias.length;                  // → undefined
```

| Operación | Array | Set |
|---|---|---|
| Añadir | `push(v)` | `add(v)` |
| Eliminar | `splice(i, 1)` | `delete(v)` |
| Tamaño | `length` | `size` |
| ¿Está? | `includes(v)` | `has(v)` |
| Duplicados | permitidos | imposibles |

## Conversiones con spread

El operador **spread** (`...`) desparrama los elementos de un iterable allí donde se escribe. Puesto entre corchetes convierte un conjunto en array:

```
const conjunto = new Set(['a', 'b', 'c']);
const lista = [...conjunto];      // → ['a', 'b', 'c']
```

Combinando las dos ideas, array → `Set` → array, se obtiene la forma más corta de **eliminar los duplicados** de un array:

```
const conRepetidos = [1, 2, 1, 3, 2];
const unicos = [...new Set(conRepetidos)];   // → [1, 2, 3]
```

Y con spread también se vuelcan varios arrays en un mismo `Set`, lo que los une sin repetidos:

```
const grupoA = ['ana', 'luis'];
const grupoB = ['luis', 'eva'];

const todos = new Set([...grupoA, ...grupoB]);
console.log(todos);           // → Set(3) { 'ana', 'luis', 'eva' }
```

▶ Por qué el orden se conserva

## Ejemplo 1: quitar los duplicados de un array

Escribe la función sinDuplicados(lista) que devuelva un nuevo array con los mismos elementos que lista pero sin repetidos, conservando el orden de la primera aparición. Resuélvelo creando un Set a partir del array y convirtiéndolo de vuelta con el operador spread.

```
function sinDuplicados(lista) {

}

console.log(sinDuplicados([3, 1, 3, 2, 1]));
```

Ctrl + Enter ejecuta con tests · Ctrl + Shift + Enter sólo el código · Tab indenta · Esc y luego Tab sale del editor

SALIDA DE CONSOLA

```
Sin salida todavía.
```

Los tests se muestran al ejecutar.

▶ Ver solución del ejemplo 1

### Ejemplo 2: unir dos arrays sin repetidos

Escribe la función union(a, b) que reciba dos arrays y devuelva un Set con todos los elementos de ambos, sin repetidos. Usa spread para volcar los dos arrays en el constructor de Set. Devuelve el Set, no un array.

```
function union(a, b) {

}

const todos = union(['ana', 'luis'], ['luis', 'eva']);
console.log(todos.size);
console.log([...todos]);
```

Ctrl + Enter ejecuta con tests · Ctrl + Shift + Enter sólo el código · Tab indenta · Esc y luego Tab sale del editor

**SALIDA DE CONSOLA**

```
Sin salida todavía.
```

Los tests se muestran al ejecutar.

▶ Ver solución del ejemplo 2